Digitalni VLSI sistemi 2025/2026

Projektni zadatak

UBRZANJE FILTRIRANJA SLIKE

Hardverski akcelerator linearnog prostorno-invarijantnog filtriranja slike

Grupa: G11

Marko Ivković 20/0596

Elektrotehnički fakultet

Školska 2025/2026. godina — 11.09.2026.

Sadržaj

Sadržaj 2
1. Uvod i opis sistema 3
1.1. Struktura sistema 4
2. Algoritam filtriranja 5
3. Arhitektura akceleratora 6
3.1. Moduli i veze između njih 6
3.2. Kontrola toka 6
3.3. Kašnjenje i protok 6
4. Opis modula 7
4.1. Aritmetička jedinica 7
4.2. Registarski interfejs 7
4.3. Zauzeće resursa 8
4.4. Struktura parametara i kvantizacija koeficijenata 9
4.5. Softverska referenca 9
4.6. Hardversko procesiranje 9
4.7. Prekidi i DMA 9
4.8. Merenje vremena (AXI Timer) 10
4.9. Verifikacija 10
4.10. Izbor filtra i tok rada 10
5. Testiranje i rezultati 11
5.1. 512x512 11
5.2. 128x128 14
5.3. Diskusija 15
6. Zaključak 15

1. Uvod i opis sistema

Filtriranje slike prostornom maskom je operacija u kojoj svaki izlazni piksel nastaje kao težinska suma piksela iz njegove okoline. Kada se radi softverski, za sliku dimenzija W×H i filtar dimenzija (2R+1)×(2R+1) procesor mora da izvrši $W \cdot H \cdot (2R+1)^2$ množenja i sabiranja, pa vreme obrade brzo raste sa veličinom slike i filtra. Ideja ovog projekta je da se taj posao prebaci na namenski hardverski akcelerator koji obradu radi protočno, približno jedan piksel po taktu, i time znatno smanji vreme izvršavanja.

Akcelerator je opisan u VHDL-u i podržava proizvoljne linearne, prostorno invarijantne filtre neparnih dimenzija, do 9×9 (radijus do 4). Ceo sistem je izgrađen oko Zynq-7000 platforme: procesorski deo (Cortex-A9) konfiguriše akcelerator preko AXI4-Lite magistrale, a same piksele slike prenosi AXI DMA kontroler preko AXI-Stream veze. Akcelerator primljene piksele filtrira i vraća ih, ponovo preko AXI-Stream-a i DMA-a, u DDR memoriju.

Završetak DMA prenosa signalizira se prekidima (MM2S i S2MM), koji se obrađuju preko kontrolera prekida (XScuGic). U izvornom kodu postoji i polling varijanta kao rezerva: ona je zadržana iz ranije faze razvoja, kada je prekidni režim pravio probleme zbog nepravilno povezanog block designa (prekidne linije DMA kontrolera nisu bile ispravno sprovedene do procesora). Nakon što je block design ispravljen, prekidi su proradili kako treba, pa se polling na kraju nije koristio. Parametar *USE_INTERRUPT* je stalno postavljen na 1, tako da se sistem uvek izvršava u prekidnom režimu. Polling bi se uključio samo rekompajliranjem sa *USE_INTERRUPT = 0.*

Za razliku od pristupa u kome je čitav akcelerator jedan veliki modul, ovde je funkcionalnost podeljena na pet manjih VHDL modula, svaki sa jasno određenom ulogom (interfejs prema magistrali, prihvat ulaznog toka, linijski bafer, aritmetika, slanje izlaza).

Provera ispravnosti se svodi na poređenje sa softverskom referencom: ista konvolucija se izračuna i na procesoru (funkcija *ImageFilterSW*) i na akceleratoru (*ImageFilterHW*), pa se rezultati uporede piksel po piksel. Softverska referenca koristi identičnu fiksno-zareznu aritmetiku kao hardver - iste kvantizovane koeficijente, isti faktor skaliranja i isto odsecanje/saturaciju - tako da se očekuje bit - tačno poklapanje rezultata. Vreme izvršavanja obe funkcije meri se pomoću AXI Timer komponente, na osnovu čega se računa faktor ubrzanja hardvera u odnosu na softver.

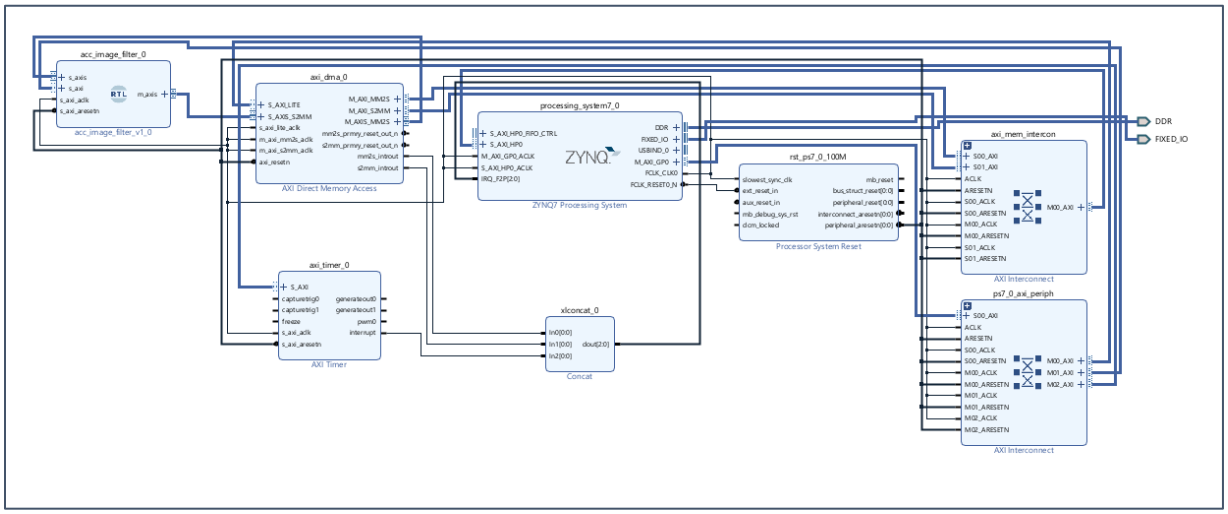
Pri svakom pokretanju korisnik iz host okruženja (XSCT) učita ulaznu sliku u DDR memoriju komandom *mwr*. Implementirana je funkcionalnost biranja tipa filtra. Po završetku, rezultujuća slika se može prebaciti nazad na host komandom *mrd*. Podržano je više tipova filtara (Box, Gausov, LoG, Sharpen, Sobel, kao i proizvoljna korisnička maska), koji se biraju bez izmene ostatka programa.

Što se obima zadatka tiče, realizovan je osnovni deo. Bonus proširenja nisu rađena: izlazna slika je manja od ulazne za 2R po obe dimenzije, jer se pikseli kojima lokalno susedstvo izlazi van okvira slike jednostavno ne računaju (nema proširenja ivica - BONUS 1, niti prilagodljive širine memorijske reči - BONUS 2).

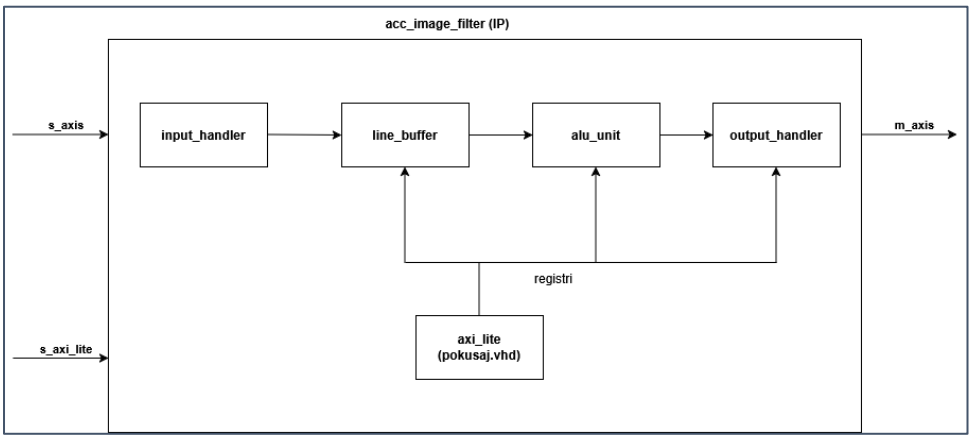
1.1. Struktura sistema

Sa strane procesorskog sistema (PS), pored Cortex-A9 procesora koriste se AXI DMA kontroler (prenos piksela ka i od akceleratora), AXI Timer (merenje vremena obrade) i kontroler prekida (obrada završetka DMA transfera). Hardverski akcelerator (acc_image_filter) je top modul u programabilnoj logici (PL) koji povezuje pet celina:

- **pokusaj** - AXI4-Lite registarski blok: procesor preko njega upisuje parametre obrade (dimenzije slike, radijus, faktor skaliranja, izlazni format i koeficijente filtra).
- **axis_input_handler** - prihvata piksele sa ulaznog AXI-Stream-a i prosleđuje ih dalje.
- **line_buffer** - čuva prethodne redove slike u BRAM-u i za svaki novi piksel formira celu vertikalnu kolonu lokalnog susedstva.
- **alu_unit** - protočna aritmetička jedinica: množi piksele koeficijentima, sabira ih, skalira i formatira izlaz.
- **axis_output_handler** - pakuje rezultat u izlazne reči i šalje ga na izlazni AXI-Stream.



Slika 1. Screenshot block design iz Vivada



Slika 2. Crtež toka podataka kroz pet modula

2. Algoritam filtriranja

Element izlazne slike predstavlja težinsku sumu kvadratnog lokalnog susedstva zadate veličine oko piksela koji se trenutno obrađuje. Za filtar radijusa R susedstvo ima dimenzije (2R+1)×(2R+1), pa je zadato $(2R+1)^2$ koeficijenata - po jedan težinski faktor za svaki piksel susedstva. Izlazna vrednost je zbir proizvoda svakog piksela susedstva sa pripadajućim koeficijentom.

Za filtar radijusa 1 (maska 3×3) potrebno je zadati 9 koeficijenata. Ako susedstvo piksela označimo kao mrežu A/B/C (redovi) i 0/1/2 (kolone), izlazne vrednosti pri obradi dva susedna piksela su:

$$B1out = W1 \cdot A0 + W2 \cdot A1 + W3 \cdot A2 + W4 \cdot B0 + W5 \cdot B1 + W6 \cdot B2 + W7 \cdot C0 + W8 \cdot C1 + W9 \cdot C2$$

$$B2out = W1 \cdot A1 + W2 \cdot A2 + W3 \cdot A3 + W4 \cdot B1 + W5 \cdot B2 + W6 \cdot B3 + W7 \cdot C1 + W8 \cdot C2 + W9 \cdot C3$$

Pikseli čije je odstojanje od okvira slike manje od radijusa filtra nemaju potpuno definisano lokalno susedstvo. U ovom radu za takve piksele se izlazna vrednost ne računa, pa je izlazna slika manja od ulazne za 2R po obe dimenzije: dimenzije izlaza su $(W-2R) \times (H-2R)$.

3. Arhitektura akceleratora

Akcelerator *acc_image_filter* je top modul koji povezuje pet funkcionalnih celina i izlaze registarskog bloka sprovodi do modula kojima su parametri potrebni. Ceo dizajn radi u jednom taktnom domenu *(s_axi_aclk*) sa zajedničkim resetom aktivnim na niskom nivou *(s_axi_aresetn*). Prema spolja, modul ima tri interfejsa: AXI4-Lite (konfiguracija), ulazni AXI4-Stream širine 8 bita (pikseli slike) i izlazni AXI4-Stream širine 16 bita.

3.1. Moduli i veze između njih

Interni tok piksela ide pravolinijski kroz četiri modula, dok registarski blok pokusaj (AXI-Lite) stoji sa strane i napaja ostale module parametrima obrade. Signali kojima su moduli povezani dati su u tabeli 1:

Tabela 1. Veze i signali između modula akceleratora

Veza	Signali	Smer
axis_input_handler -> line_buffer	pixel_data[7:0], pixel_valid / next_ready	podatak napred / spremnost nazad
line_buffer -> alu_unit	column_data[71:0], column_valid, column_en, row_start / alu_ready	kolona susedstva napred / spremnost nazad
alu_unit -> axis_output_handler	result_data[15:0], result_valid, result_last	rezultat napred
pokusaj -> svi	reg_ctrl, reg_radius, reg_img_w, reg_img_h, reg_coeff_scale, coeff_w[1295:0]	parametri obrade

3.2. Kontrola toka

Kontrola toka između modula zasniva se na paru signala „podatak validan / prijemnik spreman". Unazad kroz lanac vode signali spremnosti: *alu_ready* (od aritmetičke jedinice ka linijskom baferu) i *next_ready* (od linijskog bafera ka input handleru), koji dalje uslovljava *s_axis_tready* prema DMA kontroleru. Realizacija aritmetičke jedinice uvek prijavljuje spremnost (*alu_ready = '1'*), pošto je eksperimentalnim putem utvrđeno da komplikovaniji pristup nije neophodan.

3.3. Kašnjenje i protok

Lanac je u potpunosti protočan: svaki modul unosi fiksno kašnjenje (prijemni registar, čitanje i formiranje kolone u linijskom baferu, višestepeni pipeline aritmetičke jedinice, izlazni registar). Posle inicijalnog punjenja protoka, akcelerator daje približno jedan izlazni piksel po taktu, pa je ukupno vreme obrade određeno pre svega brojem piksela slike, a ne veličinom ili tipom filtra.

4. Opis modula

4.1. Aritmetička jedinica

Aritmetička jedinica je centralni modul akceleratora — u njoj se odvija filtriranje. Dok linijski bafer rešava vertikalnu dimenziju susedstva (dovodi celu kolonu od 9 bajtova), *alu_unit* rešava horizontalnu i obavlja aritmetiku.

Prozor susedstva: dolazne kolone se slažu u pomerački prozor od 9 kolona (*window(0..8)*), pri čemu svaka kolona nosi 9 bajtova. Time se u registrima drži kompletno dvodimenzionalno susedstvo, do maksimalnih 9×9. U svakom taktu nova kolona ulazi na poziciju *window(8),* a ostale kolone se pomeraju za jedno mesto ulevo (najstarija ispada). Na početku svakog reda slike (*row_start*) prozor se prazni i puni ispočetka. Signal *col_in_row* prati koliko je kolona stiglo u tekućem redu, čime se zna kada je horizontalno susedstvo popunjeno.

Protočna aritmetika: račun je izveden protočno, u više pipeline stepena (u cilju skraćivanja kombinacionih putanja I lakšeg zatvaranja vremenskih uslova, sistem radi na taktu od 100 MHz):

1. **Množenje** - 81 paralelnih množenja: svaki piksel susedstva (8-bitni neoznačen) množi se svojim koeficijentom (16-bitni označen).
2. **Sabiranje po grupama** - 81 proizvod se sabira u 9 grupa po 9, čime nastaje 9 parcijalnih suma.
3. **Konačno sabiranje** - 9 parcijalnih suma se sabira u jednu ukupnu sumu.
4. **Skaliranje i formatiranje** - ukupna suma se množi faktorom skaliranja (*reg_coeff_scale*), pomera, i u zavisnosti od izlaznog moda (bit MODE), formira izlaz uz saturaciju: za 8-bitni neoznačen izlaz negativne vrednosti se odsecaju na 0, a prekoračenja na 255. Za 16-bitni označen izlaz rezultat se saturira na opseg 16-bitnog broja.

Ivice i kraj slike: rezultat se prosleđuje na izlaz samo kada je susedstvo potpuno — i po vertikali (signalizira linijski bafer preko *column_valid*) i po horizontali (*col_in_row* je dostigao potrebnu vrednost). Zbog toga se ivični pikseli, kojima susedstvo izlazi iz okvira slike, ne računaju, pa je izlaz manji za 2R po obe dimenzije. Jedinica interno broji izlazne piksele i kada dostigne poslednji (*out_w−1, out_h−1*), postavlja signal *result_last*, čime akcelerator sam obeležava kraj izlaznog paketa (*tlast*) — bez oslanjanja na *tlast* sa ulaza.

4.2. Registarski interfejs

Modul *pokusaj* je AXI4-Lite slave preko koga procesor konfiguriše akcelerator. Zasnovan je na kosturu sa laboratorijskih vežbi (*acc_neg_image*), proširenom registrima specifičnim za filtar. Upis i čitanje vode dve mašine stanja sa standardnim AXI-Lite handshake-om. Adresa se dekoduje uklanjanjem donja dva bita (bajt-offset unutar reči), a preostali bitovi biraju registar.

Tabela 2. Mapa registara

Adresa	Registar	Opis
0x00	reg_ctrl	kontrolni registar; bit 0 = MODE (0 = 8-bit neoznačen, 1 = 16-bit označen)
0x04	reg_radius	radijus filtra R (1–4)
0x08	reg_img_w	širina slike
0x0C	reg_img_h	visina slike
0x10	reg_coeff_scale	faktor skaliranja
0x14 + 4*i	reg_coeff_Wi	koeficijenti filtra, i = 0..80

Napomena o rasporedu registara: tekst projektnog zadatka predviđa raspored sa 2-bajtnim razmakom, u kome je svaki registar 16-bitni, pa se dva registra smeštaju u jednu 32-bitnu reč uz upotrebu *wstrb* signala za upis pojedinačnih polovina. U ovoj realizaciji izabran je jednostavniji raspored sa 4-bajtnim razmakom (Tabela 2), gde svaki registar zauzima punu 32-bitnu reč, a koristi se samo donjih 16 bita. Iz praktičnih razloga pristup registrima na 2-bajtnim offsetima pomoću *Xil_Out32/Xil_In32* izaziva *Data Abort* na procesoru, jer te funkcije rade sa 4-bajtno poravnatim pristupom. Prelaskom na 4-bajtni raspored, problem se otklanja. Posledica je nešto širi zauzeti adresni prostor i 16 neiskorišćenih bita po reči.

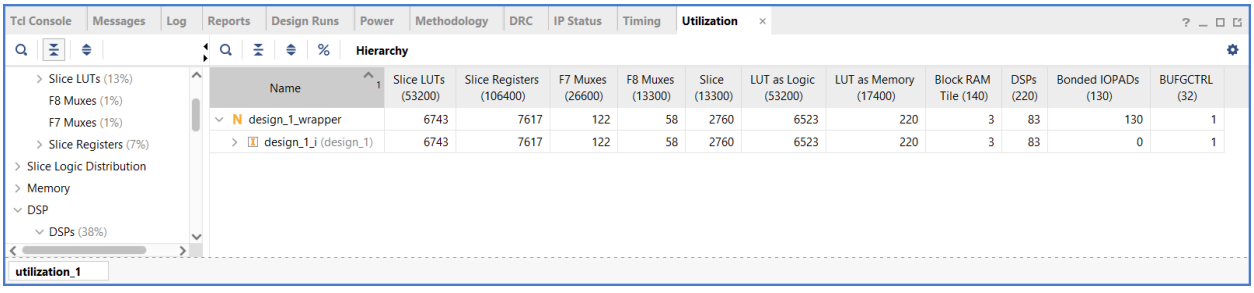
4.3. Zauzeće resursa

Tabela 3. Zauzeće resursa programabilne logike (nakon implementacije)

Resurs	Iskorišćeno	Dostupno	%
Slice LUT	6743	53200	13
Slice Registers (FF)	7617	106400	7
Block RAM Tile	3	140	2
DSP	83	220	38

Zauzeće resursa programabilne logike, dobijeno iz Vivado izveštaja o iskorišćenosti nakon implementacije, prikazano je u Tabeli 3. Najveći udeo troše DSP blokovi — 83 od 220 (≈38%) — jer aritmetička jedinica sadrži do 81 paralelno množenje (jedno po koeficijentu maske 9×9), uz nekoliko dodatnih za skaliranje. Ostali resursi su skromno zauzeti: oko 13% LUT-ova i 7% registara. Blok memorija je iskorišćena minimalno (3 od 140 BRAM blokova). Nisko zauzeće BRAM-a je posledica toga što linijski bafer čuva samo nekoliko redova slike. Dizajn je zatvorio vremenske uslove na taktu od 100 MHz (WNS > 0).

Slika 3. Screenshot utilization-a iz Vivada nakon implementacije



4.4. Struktura parametara i kvantizacija koeficijenata

Parametri jedne obrade objedinjeni su u strukturi *ProcessingParams* (dimenzije slike, radijus filtra, niz koeficijenata, faktor skaliranja, izlazni mod). Koeficijenti se u programu zadaju kao realni brojevi, a funkcija *QuantizeCoeffs* ih prevodi u celobrojni (fiksni) oblik koji hardver očekuje, i istovremeno računa celobrojnu vrednost faktora skaliranja. Ista funkcija koeficijente smešta na centar fiksne mape 9×9, čime isti raspored važi za sve radijuse.

Ključno je da se *QuantizeCoeffs* koristi na potpuno isti način i za softversku referencu i za hardversku konfiguraciju. Time obe strane rade sa identičnim koeficijentima i istim faktorom skaliranja, pa se očekuje bit - tačno poklapanje rezultata.

4.5. Softverska referenca

ImageFilterSW je referentna implementacija 2D konvolucije na procesoru. Za svaki izlazni piksel računa se suma proizvoda piksela susedstva sa odgovarajućim koeficijentima, nakon čega se rezultat množi faktorom skaliranja i formatira uz saturaciju — sve istom celobrojnom aritmetikom kao u hardveru (isto pomeranje i odsecanje, ista saturacija za oba izlazna moda). Pošto se ivični pikseli ne računaju, referentna slika je dimenzija (W−2R)×(H−2R), jednako kao hardverski izlaz. Ova funkcija služi i kao referenca za proveru ispravnosti i kao osnova za merenje softverskog vremena.

4.6. Hardversko procesiranje

Funkcija *ImageFilterHW* sprovodi celu hardversku obradu:

1. **Konfiguracija akceleratora** — kvantizovani koeficijenti, faktor skaliranja, dimenzije slike, radijus i izlazni mod upisuju se u registre akceleratora (*Xil_Out32*). Nakon upisa vrši se read-back provera ključnih registara (širina, visina, radijus) da bi se potvrdilo da je konfiguracija ispravno primljena.
2. **Priprema memorije** — izlazni bafer se popunjava kontrolnom vrednošću (marker), a zatim se keš ulaznog i izlaznog bafera usklađuje sa DDR memorijom (*Xil_DCacheFlushRange*), kako bi DMA čitao ispravne podatke.
3. **Pokretanje DMA prenosa** — prvo se pokreće prijemni (S2MM) transfer, pa tek onda predajni (MM2S), tako da je prijemna strana spremna pre nego što prvi rezultat stigne.
4. **Čekanje i završetak** — po završetku transfera keš izlaznog bafera se invalidira (*Xil_DCacheInvalidateRange),* da bi procesor pročitao stvarne rezultate iz memorije.

4.7. Prekidi i DMA

DMA kontroler radi u direktnom režimu (bez Scatter-Gather opcije). Završetak predajnog i prijemnog transfera signalizira se prekidima, koji se obrađuju preko kontrolera prekida (*XScuGic*). Prekidne rutine *TxIntrHandler i RxIntrHandler* potvrđuju prekid i postavljaju odgovarajuće zastavice (*TxDone, RxDone*), odnosno zastavicu greške. Glavni tok zatim čeka da obe zastavice budu postavljene, uz vremensko ograničenje (timeout) radi sigurnosti. Kao što je opisano u uvodu, u kodu postoji i polling varijanta kao rezerva, ali se u praksi ne koristi (*USE_INTERRUPT* je stalno 1).

4.8. Merenje vremena (AXI Timer)

Vreme izvršavanja i softverske i hardverske obrade meri se pomoću AXI Timer komponente. Tajmer se resetuje neposredno pre poziva funkcije, a po završetku se očitava i vrednost se prevodi u mikrosekunde (na osnovu učestanosti od 100 MHz). Merenjem obe funkcije nad istom slikom dobija se faktor ubrzanja hardvera u odnosu na softver, koji se ispisuje uz svaku obradu.

4.9. Verifikacija

Ispravnost hardverskog rezultata proverava se poređenjem sa referentnim (softverskim) rezultatom, piksel po piksel. Poređenje uzima u obzir izabrani izlazni mod (8-bitni neoznačeni ili 16-bitni označeni podaci). Broji se ukupan broj neslaganja i ispisuje se nekoliko prvih, radi lakšeg lociranja eventualne greške. Pošto softverska referenca koristi identičnu aritmetiku kao hardver, ispravna obrada daje potpuno (bit - tačno) poklapanje svih piksela.

4.10. Izbor filtra i tok rada

Funkcija *BuildParams* priprema koeficijente za izabrani tip filtra, zajedno sa pripadajućim radijusom, faktorom skaliranja i izlaznim modom. Time se novi filtar bira bez izmene ostatka programa.

Glavna petlja u svakom prolazu čeka na okidač koji korisnik postavlja iz host okruženja (XSCT). Radni tok je sledeći: korisnik komandom *mwr* učita ulaznu sliku u DDR memoriju i izabere tip filtra, a zatim postavljanjem okidača pokrene obradu. Program tada izračuna softversku referencu, pokrene hardversku obradu, izmeri i uporedi vremena i proveri ispravnost, i na kraju ispiše uputstvo za preuzimanje rezultujuće slike na host komandom *mrd*. Petlja se potom vraća na čekanje sledeće slike.

5. Testiranje i rezultati

5.1. 512x512

Slika 4. Box filtar za radijus R=1



Slika 5. Box filtar za radijus R=2



Slika 6. Box filtar za radijus R=3



Slika 7. Box filtar za radijus R=4



Slika 8. Sharpen filtar za radijus R=2



Slika 9. Sharpen filtar za radijus R=4



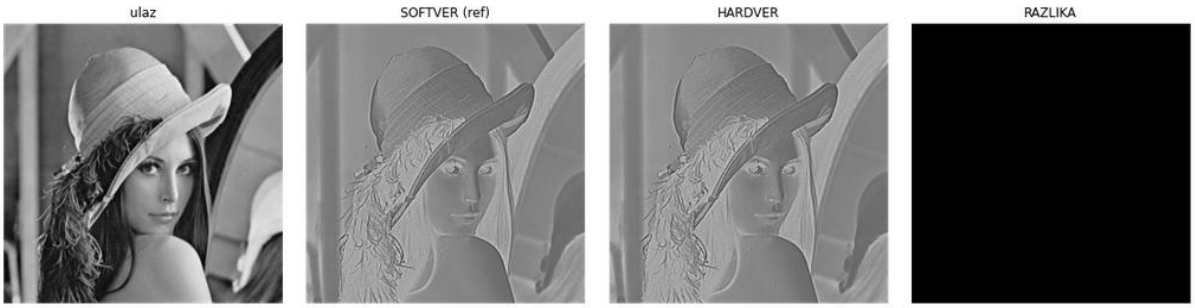
Slika 10. Gauss filtar za radijus R=2



Slika 11. Gauss filtar za radijus R=4



Slika 12. Log filtar za radijus R=2



Slika 13. Log filtar za radijus R=4

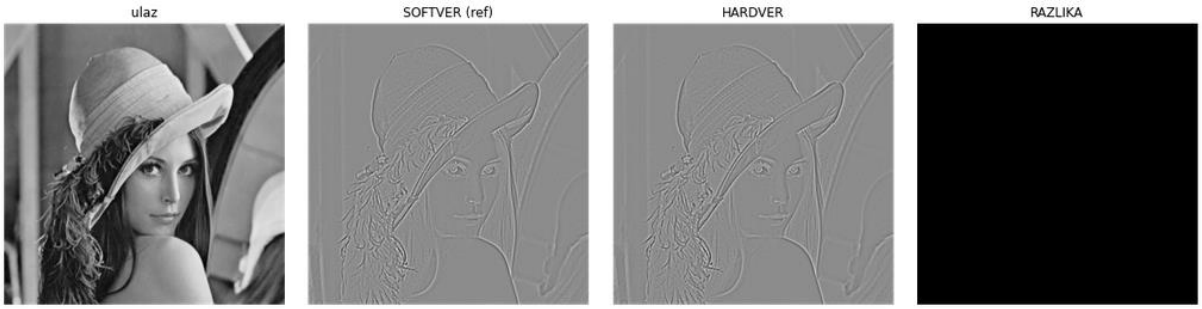
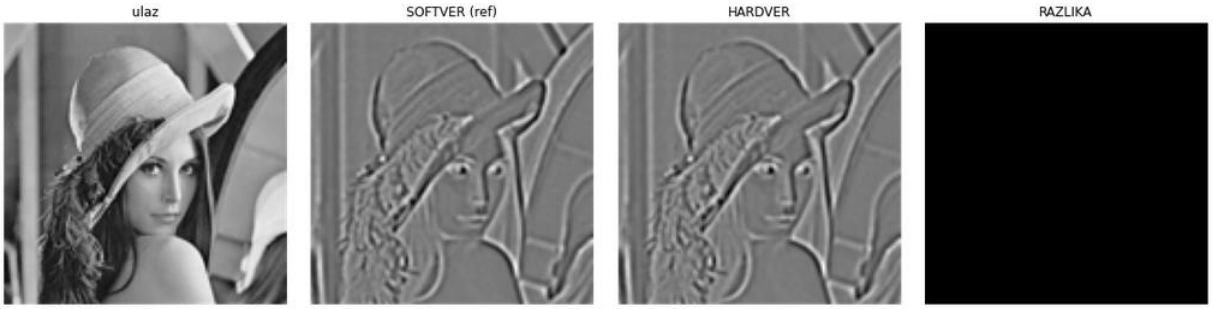


Tabela 4. Rezultati merenja korišćenjem ulazne slike dimenzija 512x512 piksela

Filtar	R	MODE	SW (µs)	HW (µs)	Ubrzanje
Box	1	0	295 263	4 369	67,6×
Box	2	0	771 379	4 364	176,8×
Box	3	0	1 421 188	4 363	325,7×
Box	4	0	2 296 329	4 346	528,4×
Gauss	2	0	771 380	4 357	177,0×
Gauss	4	0	2 296 329	4 348	528,1×
Sharpen	2	0	771 394	4 381	176,1×
Sharpen	4	0	2 296 425	4 315	532,2×
Log	2	1	774 181	5 313	145,7×
Log	4	1	2 299 050	5 329	431,4×

5.2. 128x128

Slika 14. Log filtar za radijus R=3



Slika 15. Gauss filtar za radijus R=3



Slika 16. Sharpen filtar za radijus R=3



Slika 17. Box filtar za radijus R=3



Tabela 5. Rezultati merenja korišćenjem ulazne slike dimenzija 128x128 piksela

Filtar	R	MODE	SW (µs)	HW (µs)	Ubrzanje
Box	3	0	82 628	306	270,0×
Gauss	3	0	82 628	306	270,0×
Sharpen	3	0	82 646	306	270,1×
Log	3	1	82 788	383	216,2×

5.3. Diskusija

Kod svih testiranih kombinacija hardverski rezultat se bit - tačno poklopio sa softverskom referencom, čime je potvrđeno da je model ispravan za sve filtre, radijuse i oba izlazna moda. Hardversko vreme je praktično konstantno (~4,35 ms za 512×512, ~306 µs za 128×128) i ne zavisi od tipa ni veličine filtra, jer akcelerator radi protočno - približno jedan piksel po taktu, uz paralelno množenje, pa vreme određuje broj piksela. Softversko vreme, naprotiv, raste sa kvadratom radijusa (po pikselu $(2R+1)^2$ operacija), zbog čega ubrzanje raste sa veličinom filtra: od ~68× (R=1) do preko 500× (R=4) na slici dimenzija 512×512. Filtri istog radijusa i istog moda daju gotovo identična vremena (Box, Gauss i Sharpen na R=4 svi ~528–532×), jer izvršavaju isti broj operacija. LoG, kao jedini sa 16-bitnim izlazom, prenosi dvostruko više izlaznih bajtova, pa mu je HW vreme veće a ubrzanje manje pri istom radijusu (R=2: 146× naspram ~177×).

6. Zaključak

Realizovan je i verifikovan hardverski akcelerator linearnog filtriranja slike, integrisan u kompletan sistem na Zynq-7000 platformi (AXI4-Lite konfiguracija, AXI-Stream + DMA prenos u prekidnom režimu, merenje AXI Timer-om). Ispravnost je potvrđena bit - tačnim poklapanjem sa softverskom referencom na svim filtrima i radijusima, za obe veličine slike. Merenja pokazuju da je akcelerator protočan i ograničen brojem piksela, pa je ostvareno ubrzanje u odnosu na softver od približno 68× za najmanji filtar do preko 500× za filtar 9×9 na slici 512×512, i raste sa veličinom filtra i slike. Osnovni deo zadatka je u potpunosti realizovan, bez bonus proširenja.